

Kubernetes-10 (EKS)

Aws EKS:

#####

What is EKS?

EKS is managed kubernetes service.

Meaning

Aws manages master node

Necessary app -pre installed like runtime, master process.

Manage the scaling and backup when required.

We are responsible to manage only worker nodes.

How to use eks?

We can create one eks cluster with k8s version, vpc.

Need to create node group(Group of nodes).

Attach node group to cluster and provide security group and other details.

With node group we have autoscaling.

We need to define max and min of node to nodegroup which will scale up and scale down based on load.

Finally you need to connect from your local machine to deploy applications using kubectl.

If you want to create cluster from cli then we can use eksctl tool which is provided by the community.

what makes eksctl special?

if you execute eksctl create cluster then it will use all the default value and create cluster for you.

If you want to customize then you need to modify the parameters.

We need two IAM roles to create eks cluster.

1) IAM Role for EKS Cluster -- AmazonEKSClusterPolicy

What it allows:

Communicates with other AWS services like EC2, ELB, IAM, CloudWatch.

Manages Kubernetes control plane actions like scheduling, networking, and health checks.

2) IAM Role for EKS Nodes -- AmazonEKSWorkerNodePolicy (Allows to communicate with cluster and join)

from ecr) -- AmazonEC2ContainerRegistryReadOnly (Allows to pull images
manage ips to pod) -- AmazonEKS_CNI_Policy (Allows to

Tasks:

1. Setup eks cluster using eksctl
2. setup eks cluster using console
3. Setup HPA
4. Setup cluster autoscale
5. Setup cronjob and job
6. Create secret and inject inside pod
7. Check differen add-ons available on eks.
8. Upgrade eks cluster

TASK 1 – Create EKS Cluster using eksctl (us-east-2)

We are creating cluster in:

Amazon Elastic Kubernetes Service (EKS)

Before Running Anything – Quick Check

Run these commands: in local terminal

```
aws --version
kubectl version --client
eksctl version
```

```
0.223.0-dev+7a9410b8d.2026-02-14T08:10:24Z
[waseemakram@waseem Downloads % aws --version
aws-cli/2.31.39 Python/3.13.9 Darwin/25.3.0 source/arm64
[waseemakram@waseem Downloads % eksctl version
0.223.0-dev+7a9410b8d.2026-02-14T08:10:24Z
[waseemakram@waseem Downloads % kubectl version --client
Client Version: v1.35.1
Kustomize Version: v5.7.1
waseemakram@waseem Downloads %
```

Now Let's Create the Cluster

Step 1 – Confirm Region

Run:

```
aws configure list
Make sure region = us-east-2
```

```
[waseemakram@waseem eks % aws configure
AWS Access Key ID [*****MYYF]:
AWS Secret Access Key [*****Z05I]:
Default region name [us-east-1]: us-east-2
Default output format [json]:
waseemakram@waseem eks %
```

Step 2 – Create YAML File

In your terminal:

Vi eks-free-tier.yaml

```
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: free-tier-eks
  region: us-east-2
  version: "1.28"

vpc:
  clusterEndpoints:
    publicAccess: true
    privateAccess: false
  publicAccessCidrs:
    - 0.0.0.0/0

nodeGroups:
  - name: free-tier-nodes
    instanceType: t3.small
    desiredCapacity: 2
    minSize: 2
    maxSize: 3
    volumeSize: 20
    iam:
      withAddonPolicies:
        autoScaler: true
        ebs: true
        albIngress: true
```

~

Step 3 – Create Cluster

Now run:

```
eksctl create cluster -f eks-free-tier.yaml
```

Important:

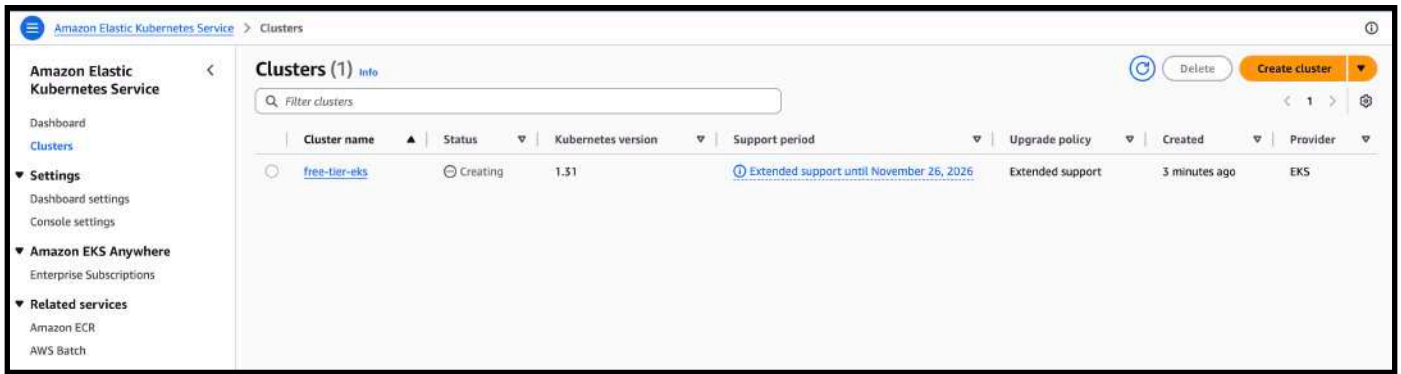
This will take 15–20 minutes.

You will see logs like:

- creating VPC
- creating cluster
- creating nodegroup
- waiting for nodes

Let it run. Don't interrupt.

```
waseemakram@waseem eks % eksctl create cluster -f eks-free-tier.yaml
2026-02-28 14:19:07 [i] nodegroup.iam.withAddonPolicies.albIngress field is deprecated, please use awsLoadBalancerController instead
2026-02-28 14:19:07 [i] eksctl version 0.223.0-dev+7a9410b8d.2026-02-14T08:10:24Z
2026-02-28 14:19:07 [i] using region us-east-2
2026-02-28 14:19:09 [i] setting availability zones to [us-east-2c us-east-2a us-east-2b]
2026-02-28 14:19:09 [i] subnets for us-east-2c - public:192.168.0.0/19 private:192.168.96.0/19
2026-02-28 14:19:09 [i] subnets for us-east-2a - public:192.168.32.0/19 private:192.168.128.0/19
2026-02-28 14:19:09 [i] subnets for us-east-2b - public:192.168.64.0/19 private:192.168.160.0/19
2026-02-28 14:19:10 [i] nodegroup "free-tier-nodes" will use "ami-0bdc1d1a0a4a28cb4" [AmazonLinux2023/1.31]
2026-02-28 14:19:10 [i] Auto Mode will be enabled by default in an upcoming release of eksctl. This means managed node groups and managed networking add-ons will no longer be created by default. To maintain current behavior, explicitly set 'autoModeConfig.enabled: false' in your cluster configuration. Learn more: https://eksctl.io/usage/auto-mode/
2026-02-28 14:19:10 [i] using Kubernetes version 1.31
2026-02-28 14:19:10 [i] creating EKS cluster "free-tier-eks" in "us-east-2" region with un-managed nodes
2026-02-28 14:19:10 [i] 1 nodegroup (free-tier-nodes) was included (based on the include/exclude rules)
2026-02-28 14:19:10 [i] will create a CloudFormation stack for cluster itself and 1 nodegroup stack(s)
2026-02-28 14:19:10 [i] if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=us-east-2 --cluster=free-tier-eks'
2026-02-28 14:19:10 [i] Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "free-tier-eks" in "us-east-2"
2026-02-28 14:19:10 [i] CloudWatch logging will not be enabled for cluster "free-tier-eks" in "us-east-2"
2026-02-28 14:19:10 [i] you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=us-east-2 --cluster=free-tier-eks'
2026-02-28 14:19:10 [i] default addons metrics-server, vpc-cni, kube-proxy, coredns were not specified, will install them as EKS addons
2026-02-28 14:19:10 [i]
2 sequential tasks: { create cluster control plane "free-tier-eks",
  2 sequential sub-tasks: {
    2 sequential sub-tasks: {
      1 task: { create addons },
      wait for control plane to become ready,
    },
    create nodegroup "free-tier-nodes",
  }
}
2026-02-28 14:19:10 [i] building cluster stack "eksctl-free-tier-eks-cluster"
2026-02-28 14:19:12 [i] deploying stack "eksctl-free-tier-eks-cluster"
```



—> wait until status ACTIVE

While status = **Creating**

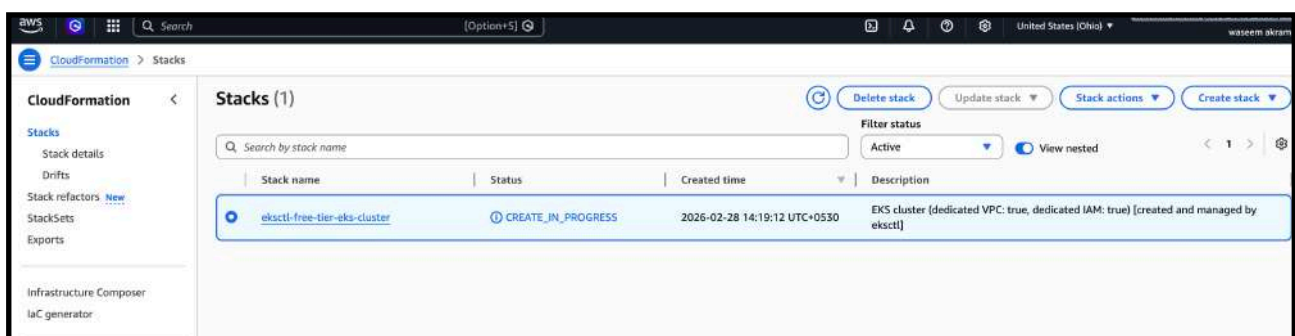
AWS is:

- Creating VPC
- Creating subnets
- Creating security groups
- Creating EKS control plane
- Creating Node Group (EC2 instances)
- Attaching IAM roles
- Configuring networking

This takes around **15–20 minutes**.

Do NOT cancel.

—> also check cloud formation



CloudFormation > **Stacks** > eksctl-free-tier-eks-cluster

Stacks (1)

Filter status: Active

View nested

Stacks

- eksctl-free-tier-eks-cluster
 - 2026-02-28 14:19:12 UTC+0530
 - CREATE_IN_PROGRESS

Events (98)

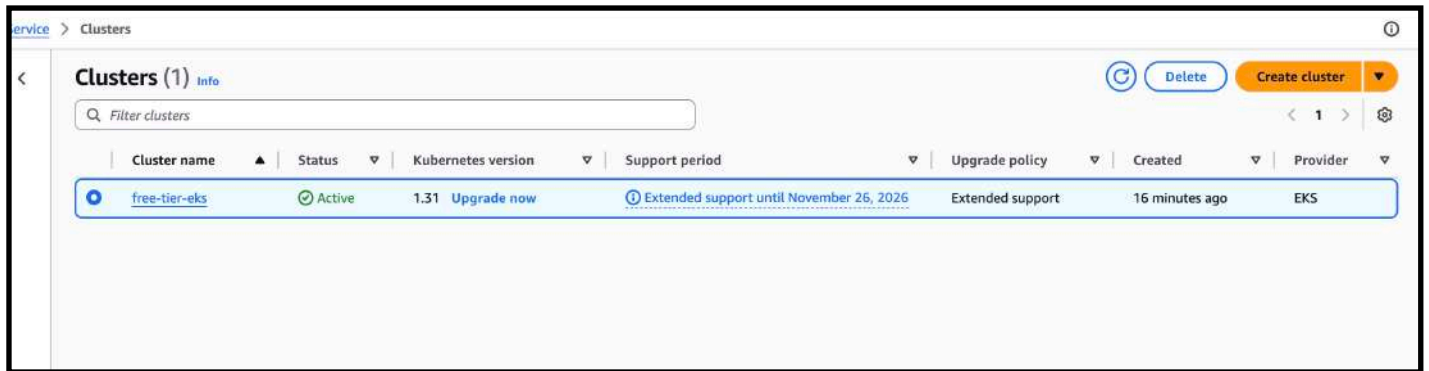
Operation ID	Timestamp	Logical ID	Status	Detailed status
22202ec4-469e-4bb3-a7ce-28e1525b1dba	2026-02-28 14:21:29 UTC+0530	NATPrivateSubnetRoute USEAST2C	CREATE_COMPLETE	
22202ec4-469e-4bb3-a7ce-28e1525b1dba	2026-02-28 14:21:29 UTC+0530	NATPrivateSubnetRoute USEAST2B	CREATE_COMPLETE	
22202ec4-469e-4bb3-a7ce-28e1525b1dba	2026-02-28 14:21:29 UTC+0530	NATPrivateSubnetRoute USEAST2A	CREATE_COMPLETE	
22202ec4-469e-4bb3-a7ce-28e1525b1dba	2026-02-28 14:21:28 UTC+0530	NATPrivateSubnetRoute USEAST2C	CREATE_IN_PROGRESS	
22202ec4-469e-4bb3-a7ce-28e1525b1dba	2026-02-28 14:21:28 UTC+0530	NATPrivateSubnetRoute USEAST2B	CREATE_IN_PROGRESS	
22202ec4-469e-4bb3-a7ce-28e1525b1dba	2026-02-28 14:21:28 UTC+0530	NATPrivateSubnetRoute USEAST2A	CREATE_IN_PROGRESS	
22202ec4-469e-4bb3-a7ce-28e1525b1dba	2026-02-28 14:21:27 UTC+0530	NATPrivateSubnetRoute USEAST2B	CREATE_IN_PROGRESS	
22202ec4-469e-4bb3-a7ce-28e1525b1dba	2026-02-28 14:21:27 UTC+0530	NATPrivateSubnetRoute USEAST2C	CREATE_IN_PROGRESS	

—> wait until created

```

2026-02-28 14:27:21 [I] waiting for CloudFormation stack "eksctl-free-tier-eks-cluster"
2026-02-28 14:28:22 [I] waiting for CloudFormation stack "eksctl-free-tier-eks-cluster"
2026-02-28 14:28:27 [I] recommended policies were found for "vpc-cni" addon, but since DDC is disabled on the cluster, eksctl cannot configure the requested permissions; the recommended way to provide IAM permissions for "vpc-cni" addon is via pod identity associations; after addon creation is completed, add all recommended policies to the config file, under 'addon.PodIdentityAssociations', and run 'eksctl update addon'
2026-02-28 14:28:27 [I] creating addon: vpc-cni
2026-02-28 14:28:28 [I] successfully created addon: vpc-cni
2026-02-28 14:28:29 [I] creating addon: kube-proxy
2026-02-28 14:28:29 [I] successfully created addon: kube-proxy
2026-02-28 14:28:30 [I] creating addon: coreDNS
2026-02-28 14:28:30 [I] successfully created addon: coreDNS
2026-02-28 14:30:35 [I] building nodegroup stack "eksctl-free-tier-eks-nodegroup-free-tier-nodes"
2026-02-28 14:30:37 [I] deploying stack "eksctl-free-tier-eks-nodegroup-free-tier-nodes"
2026-02-28 14:30:38 [I] waiting for CloudFormation stack "eksctl-free-tier-eks-nodegroup-free-tier-nodes"
2026-02-28 14:31:09 [I] waiting for CloudFormation stack "eksctl-free-tier-eks-nodegroup-free-tier-nodes"
2026-02-28 14:31:54 [I] waiting for CloudFormation stack "eksctl-free-tier-eks-nodegroup-free-tier-nodes"
2026-02-28 14:33:35 [I] waiting for the control plane to become ready
2026-02-28 14:33:36 [I] saved kubeconfig as "/Users/waseemakram/.kube/config"
2026-02-28 14:33:36 [I] no tasks
2026-02-28 14:33:36 [I] all EKS cluster resources for "free-tier-eks" have been created
2026-02-28 14:33:37 [I] nodegroup "free-tier-nodes" has 2 node(s)
2026-02-28 14:33:37 [I] node "ip-192-168-27-228.us-east-2.compute.internal" is not ready
2026-02-28 14:33:37 [I] node "ip-192-168-32-33.us-east-2.compute.internal" is not ready
2026-02-28 14:33:37 [I] waiting for at least 2 node(s) to become ready in "free-tier-nodes"
2026-02-28 14:33:37 [I] nodegroup "free-tier-nodes" has 2 node(s)
2026-02-28 14:33:45 [I] node "ip-192-168-27-228.us-east-2.compute.internal" is ready
2026-02-28 14:33:45 [I] node "ip-192-168-32-33.us-east-2.compute.internal" is ready
2026-02-28 14:33:45 [I] created 1 nodegroup(s) in cluster "free-tier-eks"
2026-02-28 14:33:47 [I] creating addon: metrics-server
2026-02-28 14:33:47 [I] successfully created addon: metrics-server
2026-02-28 14:33:49 [I] kubect command should work with "/Users/waseemakram/.kube/config", try 'kubectl get nodes'
2026-02-28 14:33:49 [I] EKS cluster "free-tier-eks" in "us-east-2" region is ready
waseemakram@waseem eks %
  
```

—> now its successfully created



When Status Changes to ACTIVE

Run this in your terminal:

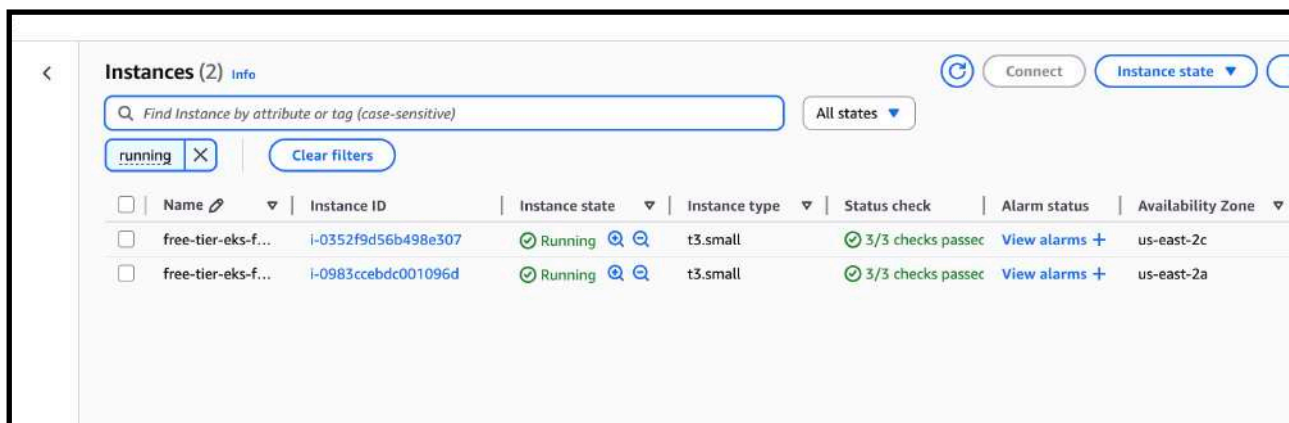
```
kubectl get nodes
```

If you see 2 nodes in **Ready** state → Cluster fully ready

```
waseemakram@waseem eks % kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-27-228.us-east-2.compute.internal	Ready	<none>	2m43s	v1.31.14-eks-efcacff
ip-192-168-32-33.us-east-2.compute.internal	Ready	<none>	2m40s	v1.31.14-eks-efcacff

```
waseemakram@waseem eks %
```



```

ip-192-168-32-33.us-east-2.compute.internal    Ready    <none>    2m40s    v1.31.14-eks-ec2c11
waseemakram@waseem eks % kubectl get pods -A

```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	aws-node-j7rb2	2/2	Running	0	5m11s
kube-system	aws-node-mmbsv	2/2	Running	0	5m14s
kube-system	coredns-6b94694fcb-blkbh	1/1	Running	0	10m
kube-system	coredns-6b94694fcb-f9pvr	1/1	Running	0	10m
kube-system	kube-proxy-hgzdq	1/1	Running	0	5m14s
kube-system	kube-proxy-mjbvm	1/1	Running	0	5m11s
kube-system	metrics-server-6899488d4b-kw7pd	1/1	Running	0	4m51s
kube-system	metrics-server-6899488d4b-qtt5b	1/1	Running	0	4m51s

```

waseemakram@waseem eks %

```

TASK 2 – Create EKS Cluster Using AWS Console

Step 1 – Open EKS

1. Login to AWS Console
2. Make sure region = **us-east-2** (top right corner) because iam using that region
3. Search → **EKS**
4. Click **Clusters**
5. Click **Create cluster**

Step 2 – Configure Cluster

Choose:

Cluster configuration:

- Cluster name:

console-eks

- Kubernetes version:

1.31

- Cluster service role:
If dropdown shows a role like:

eksctl-free-tier-eks-cluster-ServiceRole
You can use it.

If no role:

Click **Create recommended role**.

Click **Next**

Amazon Elastic Kubernetes Service > Create EKS cluster

Introducing Amazon EKS Provisioned control plane
You can now choose a control plane scaling tier with the capacity required for your workload to ensure predictable and high performance. [Learn more](#)

Step 1: Configure cluster

Configure cluster

Configuration options - new
Choose how you would like to configure the cluster.

- ☐ Quick configuration (with EKS Auto Mode) - new
Quickly create a cluster with production-grade default settings. The configuration uses EKS Auto Mode to automate infrastructure tasks like creating nodes and provisioning storage.
- ☒ Custom configuration
To change default settings prior to creation, choose this option. This configuration gives the option to use EKS Auto Mode and customize the cluster's configuration.

EKS Auto Mode - new
Choose if you would like to use EKS's Auto Mode.

- ☒ Use EKS Auto Mode
EKS automates routine cluster tasks for compute, storage, and networking. When a new pod can't fit onto existing nodes, EKS creates a new node. EKS combines cluster infrastructure managed by AWS with integrated Kubernetes capabilities to meet application compute needs. [View pricing](#)

Cluster configuration

Name
Enter a unique name for this cluster. This property cannot be changed after the cluster is created.
console-eks
The cluster name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 100.

Cluster IAM role
Select the Cluster IAM role to allow the Kubernetes control plane to manage AWS resources on your behalf. This cannot be changed after the cluster is created. To create a new custom role, follow the instructions in the [Amazon EKS User Guide](#).
eksctl-free-tier-eks-cluster-ServiceRole-OnC5leZfV8MF [Create recommended role](#)

Step 3 – Networking

For learning, keep it simple:

- VPC → Select Default VPC
- Subnets → Select all available subnets
- Cluster endpoint access:
 - Enable Public
 - Disable Private

Click **Next**

Step 1
● Configure cluster

Step 2
● **Specify networking**

Step 3
○ Configure observability

Step 4
○ Select add-ons

Step 5
○ Configure selected add-ons settings

Step 6
○ Review and create

Specify networking

Networking [Info](#)
IP address family and service IP address range cannot be changed after cluster creation.

VPC [Info](#)
Select a VPC to use for your EKS cluster resources.

vpc-021bc537bb7f21263 | Default ▼

Subnets [Info](#)
Choose the subnets in your VPC where the control plane may place elastic network interfaces (ENIs) to facilitate communication with your cluster. To create a new subnet, click [Create Subnet](#).

Select subnets ▼

subnet-0551f35a3dc0f9911 us-east-2a 172.31.0.0/20 Type: Public ✕	subnet-03bce592373e42366 mypublic-vpc-default ✕ us-east-2c 172.31.32.0/20 Type: Public
subnet-0070d56ed6dd106b5 ✕ us-east-2b 172.31.16.0/20 Type: Public	

Additional security groups [Info](#)
EKS automatically creates a cluster security group on cluster creation to facilitate communication between worker nodes and control plane. Optionally, choose interfaces that are created in your control plane subnets. To create a new security group, go to the corresponding page in the [VPC console](#).

Select security groups ▼

sg-0e76175cb35964c64 | default ✕
default VPC security group

Choose cluster IP address family [Info](#)
Specify the IP address type for pods and services in your cluster.

Step 4 – Configure Logging (Optional)

You can skip logging for now.

Click **Next**

Step 5 – Review and Create

Click **Create**



Wait 10–15 minutes until status becomes:

ACTIVE

Amazon Elastic Kubernetes Service > Clusters > console-eks

Amazon Elastic Kubernetes Service

Dashboard
Clusters

▼ Settings
Dashboard settings
Console settings

▼ Amazon EKS Anywhere
Enterprise Subscriptions

▼ Related services
Amazon ECR
AWS Batch

Documentation
EKS workshops

console-eks

The Amazon EKS console does not show Kubernetes resources until the cluster has an ACTIVE or UPDATING state. Please try again in a few minutes.

▼ **Cluster info**

Status Creating	Kubernetes version 1.31	Support period Extended support until November 26, 2026	Provider EKS
Cluster health 0	Upgrade insights 0	Node health issues 0	Capability issues 0

Overview Resources Compute Networking Add-ons Capabilities Access Observability Update history & backups Tags

Details

API server endpoint -	OpenID Connect provider URL -	Created a few seconds ago
Certificate authority -	Cluster IAM role ARN arn:aws:iam::207662791773:role/eksctl-free-tier-eks-cluster-ServiceRole-OnC5leZIV8MF View in IAM	Cluster ARN arn:aws:eks:us-east-2:207662791773:cluster/console-eks
		Platform version eks.52

—> after active

Clusters (2)

Filter clusters

Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created
console-eks	Active	1.31 Upgrade now	Extended support until November 26, 2026	Extended support	12 minutes ago
free-tier-eks	Active	1.31 Upgrade now	Extended support until November 26, 2026	Extended support	an hour ago

Kubernetes Service > Clusters > console-eks

console-eks

End of extended support for Kubernetes version (1.31) is November 26, 2026.

▼ **Cluster info**

Status Active	Kubernetes version 1.31	Support period Extended support until November 26, 2026	Provider EKS
Cluster health 0	Upgrade insights 0	Node health issues 0	Capability issues 0

Overview Resources Compute Networking Add-ons Capabilities Access Observability Update history & backups Tags

Details

API server endpoint https://CD2176317825099D69A42033F60CE57B.gr7.us-east-2.eks.amazonaws.com	OpenID Connect provider URL https://oidc.eks.us-east-2.amazonaws.com/id/CD2176317825099D69A42033F60CE57B	Created 13 minutes ago
Certificate authority LS0tLS1CRUdJTiBDRVJUSUZJQ0FUR5OktLS0tCk1JSURCVENRNDQWUyZDZ3SUJBZ0UZE93SVpudHE1VHN3RFFZSkVWklodmNOQVFFTEJRQXdGVEVUTUJFR0ExVUUKQXhNS2Ez	Cluster IAM role ARN arn:aws:iam::207662791773:role/eksctl-free-tier-eks-cluster-ServiceRole-OnC5leZIV8MF View in IAM	Cluster ARN arn:aws:eks:us-east-2:207662791773:cluster/console-eks
		Platform version eks.52

Next step: **Create Node Group** (very important).

Right now cluster has only control plane.
No worker nodes yet → No pods can run.

Create Node Group (Console)

We are still inside:

Amazon Elastic Kubernetes Service

Step 1 – Go to Compute Tab

Inside **console-eks**:

1. Click **Compute** tab
2. Click **Add node group**

Step 2 – Configure Node Group

Fill details:

Node group name:

`console-nodegroup`

Node IAM role:

If dropdown shows existing Node role (like from eksctl cluster), select it.

If not available:

Click **Create recommended role**

Click **Next**

Amazon Elastic Kubernetes Service > Clusters > console-eks > Add node group

Cluster creation request submitted
console-eks is being created. Learn how to [deploy a sample app](#). EKS will create nodes as needed.

Notifications 0 0 0 0 2 0

Step 1
● **Configure node group**
Step 2
○ Set compute and scaling configuration
Step 3
○ Specify networking
Step 4
○ Review and create

Configure node group [info](#)

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add multiple node groups to your cluster.

Node group configuration

These properties cannot be changed after the node group is created.

Name
Assign a unique name for this node group.
console-nodegroup

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 63.

Node IAM role [info](#)
Select the IAM role that will be used by the nodes. To create a new role, go to the [IAM console](#).

eksctl-free-tier-eks-nodegroup-fre-NodeInstanceRole-H6ZuARdZzSEu [Create recommended role](#)

The selected role must not be used by a self-managed node group as this could lead to a service interruption upon managed node group deletion.
[Learn more](#)

Launch template [info](#)

These properties cannot be changed after the node group is created.

☒ **Use launch template**
Configure this node group using an EC2 launch template.

Step 3 – Set Compute Configuration

Instance type:

t3.small

Scaling configuration:

- Desired size: 2
- Minimum size: 2
- Maximum size: 3

Click **Next**

Amazon Elastic Kubernetes Service > Clusters > console-eks > Add node group

Cluster creation request submitted
console-eks is being created. Learn how to [deploy a sample app](#). EKS will create nodes as needed.

Notifications 0 0 0 0 2 0

Step 1
● Configure node group
Step 2
● **Set compute and scaling configuration**
Step 3
○ Specify networking
Step 4
○ Review and create

Set compute and scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type [info](#)
Select the EKS-optimized Amazon Machine Image for nodes.
Amazon Linux 2023 (x86_64) Standard (AL2023_x86_64_STANDARD)

Capacity type
Select the capacity purchase option for this node group.
On-Demand

Instance types [info](#)
Select instance types you prefer for this node group.
Enter an instance type:
t3.small vCPU: 2 vCPUs Memory: 2 GiB Network: Up to 5 Gigabit Max ENI: 5 Max IPs: 12

Disk size
Select the size of the attached EBS volume for each node.
20 GiB

Node group scaling configuration

Desired size
Set the desired number of nodes that the group should launch with initially.
2 nodes

Step 4 – Networking

Select all subnets (default ones).

Click **Next**

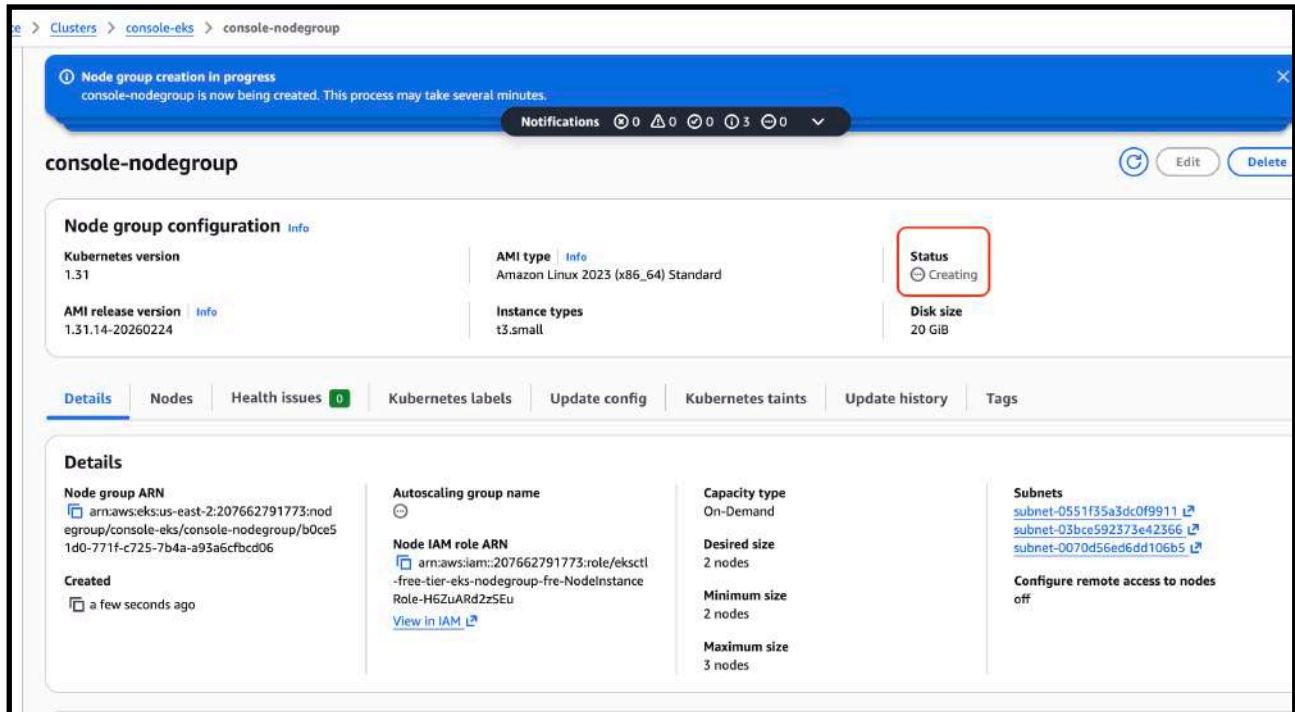
Step 5 – Review and Create

Click **Create**

 Wait 5–10 minutes.

Status will become:

Active



The screenshot shows the AWS Management Console interface for a Kubernetes node group named 'console-nodegroup'. At the top, a blue notification bar states 'Node group creation in progress' and 'console-nodegroup is now being created. This process may take several minutes.' Below this, the 'console-nodegroup' header is visible with 'Edit' and 'Delete' buttons. The main configuration section displays the following details:

- Node group configuration**
 - Kubernetes version: 1.31
 - AMI type: Amazon Linux 2023 (x86_64) Standard
 - AMI release version: 1.31.14-20260224
 - Instance types: t3.small
 - Status**: Creating (highlighted with a red box)
 - Disk size: 20 GiB

Below the configuration, a series of tabs are shown: Details, Nodes, Health issues (0), Kubernetes labels, Update config, Kubernetes taints, Update history, and Tags. The 'Details' tab is selected, showing the following information:

- Node group ARN**: arn:aws:eks:us-east-2:207662791773:nodegroup/console-eks/console-nodegroup/b0ce51d0-771f-c725-7b4a-a93a5cfbcd06
- Created**: a few seconds ago
- Autoscaling group name**: [icon]
- Node IAM role ARN**: arn:aws:iam::207662791773:role/eksctl-free-tier-eks-nodegroup-fre-NodeInstanceRole-H6ZuARd2zSEu (with a 'View in IAM' link)
- Capacity type**: On-Demand
- Desired size**: 2 nodes
- Minimum size**: 2 nodes
- Maximum size**: 3 nodes
- Subnets**: subnet-0551f35a3dc0f9911, subnet-03bce592373e42366, subnet-0070d56ed6dd106b5 (each with a link icon)
- Configure remote access to nodes**: off

After Node Group Active

We test it.

Run this in terminal:

```
aws eks update-kubeconfig --region us-east-2 --name console-eks
```

Then:

```
kubectl get nodes
```

Task 3: Setup HPA (Horizontal Pod Autoscaler)

We will do this on your **CLI cluster** → `free-tier-eks`

First confirm you're connected to correct cluster:

```
kubectl config current-context
```

It should show something like:

```
free-tier-eks
```

```
waseemakram@waseem eks % kubectl config current-context
iam-root-account@free-tier-eks.us-east-2.eksctl.io
waseemakram@waseem eks % kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-27-228.us-east-2.compute.internal	Ready	<none>	79m	v1.31.14-eks-efcacff
ip-192-168-32-33.us-east-2.compute.internal	Ready	<none>	79m	v1.31.14-eks-efcacff

```
waseemakram@waseem eks %
```

Step 1 – Create Nginx Deployment

vi nginx-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
        resources:
          requests:
            cpu: 100m
          limits:
            cpu: 200m
        ports:
        - containerPort: 80
```

Apply:

kubectl apply -f nginx-deployment.yaml

```
waseemakram@waseem eks % kubectl apply -f nginx-deployment.yaml  
  
deployment.apps/nginx-deployment created  
waseemakram@waseem eks % kubectl get pods  
  
NAME                                READY   STATUS    RESTARTS   AGE  
nginx-deployment-66f5cbd4b-lrwtx   1/1     Running   0           6s  
waseemakram@waseem eks %
```

STEP 2 – Create NodePort Service

vi nginx-nodeport-service.yaml

```
waseemakram@waseem eks % vi nginx-nodeport-service.yaml  
  
[waseemakram@waseem eks % cat nginx-nodeport-service.yaml  
apiVersion: v1  
kind: Service  
metadata:  
  name: nginx-service  
spec:  
  type: NodePort  
  selector:  
    app: nginx  
  ports:  
    - port: 80  
      targetPort: 80  
      nodePort: 30080  
  
waseemakram@waseem eks %
```

Save and apply:

```
kubectl apply -f nginx-nodeport-service.yaml
```

Check:

```
kubectl get svc
```

```
waseemakram@waseem eks % kubectl apply -f nginx-nodeport-service.yaml
service/nginx-service created
waseemakram@waseem eks % kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	98m
nginx-service	NodePort	10.100.137.96	<none>	80:30080/TCP	41s

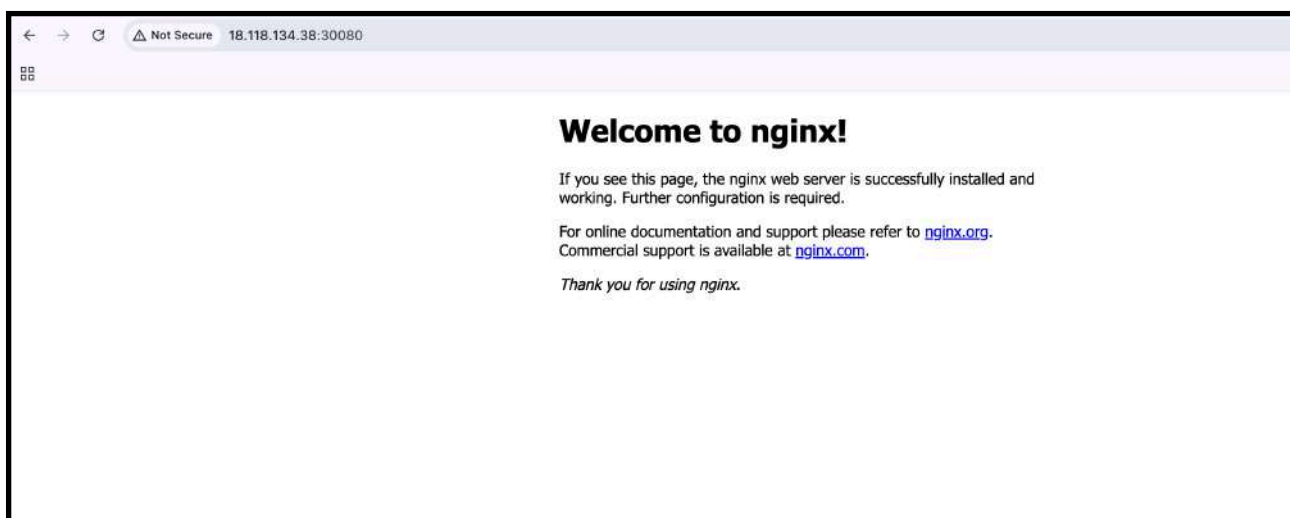
```
waseemakram@waseem eks %
```

Get node public IP from EC2.

Open browser:

```
http://<EC2-PUBLIC-IP>:30080
```

You should see nginx page.



STEP 3 – Create HPA (Target 30%)

vi hpa.yaml

```
waseemakram@waseem eks % vi hpa.yaml
waseemakram@waseem eks % cat hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 30
waseemakram@waseem eks %
```

Apply:

kubectl apply -f hpa.yaml

Check:

kubectl get hpa

```
waseemakram@waseem eks % kubectl apply -f hpa.yaml
horizontalpodautoscaler.autoscaling/nginx-hpa created
waseemakram@waseem eks % kubectl get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
nginx-hpa	Deployment/nginx-deployment	cpu: <unknown>/30%	1	5	0	9s

```
waseemakram@waseem eks %
```

STEP 4 – Create Load Generator

```
vi load.yaml
```

```
waseemakram@waseem eks % vi load.yaml
waseemakram@waseem eks % cat load.yaml
apiVersion: v1
kind: Pod
metadata:
  name: load-generator
spec:
  containers:
  - name: busybox
    image: busybox
    command: ["/bin/sh"]
    args:
      - -c
      - while true; do wget -q -O- http://nginx-service; done
waseemakram@waseem eks %
```

Apply:

```
kubectl apply -f load.yaml
```

STEP 5 – Watch Scaling

Run:

```
kubectl get hpa -w  
and
```

```
kubectl get pods -w  
Within 1–3 minutes:
```

You should see:

REPLICAS: 2

```
waseemakram@waseem eks % kubectl apply -f load.yaml  
pod/load-generator created  
waseemakram@waseem eks % kubectl get hpa -w  


| NAME      | REFERENCE                   | TARGETS     | MINPODS | MAXPODS | REPLICAS | AGE  |
|-----------|-----------------------------|-------------|---------|---------|----------|------|
| nginx-hpa | Deployment/nginx-deployment | cpu: 0%/30% | 1       | 5       | 1        | 101s |

  
waseemakram@waseem eks % kubectl get hpa -w  


| NAME      | REFERENCE                   | TARGETS      | MINPODS | MAXPODS | REPLICAS | AGE   |
|-----------|-----------------------------|--------------|---------|---------|----------|-------|
| nginx-hpa | Deployment/nginx-deployment | cpu: 25%/30% | 1       | 5       | 1        | 2m19s |

  
waseemakram@waseem eks % kubectl get pods -w  


| NAME                             | READY | STATUS  | RESTARTS | AGE |
|----------------------------------|-------|---------|----------|-----|
| load-generator                   | 1/1   | Running | 0        | 58s |
| nginx-deployment-66f5cbd4b-gq8lq | 1/1   | Running | 0        | 7s  |
| nginx-deployment-66f5cbd4b-lrwtx | 1/1   | Running | 0        | 12m |

  
waseemakram@waseem eks % kubectl get hpa -w  


| NAME      | REFERENCE                   | TARGETS      | MINPODS | MAXPODS | REPLICAS | AGE   |
|-----------|-----------------------------|--------------|---------|---------|----------|-------|
| nginx-hpa | Deployment/nginx-deployment | cpu: 35%/30% | 1       | 5       | 2        | 2m58s |
| nginx-hpa | Deployment/nginx-deployment | cpu: 29%/30% | 1       | 5       | 2        | 3m    |

  
waseemakram@waseem eks % kubectl get pods -w  


| NAME                             | READY | STATUS  | RESTARTS | AGE |
|----------------------------------|-------|---------|----------|-----|
| load-generator                   | 1/1   | Running | 0        | 95s |
| nginx-deployment-66f5cbd4b-gq8lq | 1/1   | Running | 0        | 44s |
| nginx-deployment-66f5cbd4b-lrwtx | 1/1   | Running | 0        | 12m |


```

Initially:

cpu: 0%/30%

REPLICAS: 1

Then:

cpu: 25%/30%

REPLICAS: 1

Then:

cpu: 35%/30%

REPLICAS: 2

—> CPU crossed 30%

—> HPA increased replicas from 1 → 2

And in kubectl get pods -w:

You can clearly see:

nginx-deployment-...-gq8lq

nginx-deployment-...-lrwtx

Two nginx pods running.

STEP 6 – Delete Load

kubectl delete pod load-generator

Wait 2–3 minutes.

```
waseemakram@waseem eks % kubectl delete pod load-generator
pod "load-generator" deleted from default namespace
waseemakram@waseem eks % kubectl get hpa
kubectl get pods
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
nginx-hpa	Deployment/nginx-deployment	cpu: 30%/30%	1	5	2	6m54s

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-66f5cbd4b-gq8lq	1/1	Running	0	4m26s
nginx-deployment-66f5cbd4b-lrwtx	1/1	Running	0	16m

```
waseemakram@waseem eks % kubectl get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
nginx-hpa	Deployment/nginx-deployment	cpu: 0%/30%	1	5	2	7m34s

```
waseemakram@waseem eks % kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-66f5cbd4b-gq8lq	1/1	Running	0	5m19s
nginx-deployment-66f5cbd4b-lrwtx	1/1	Running	0	17m

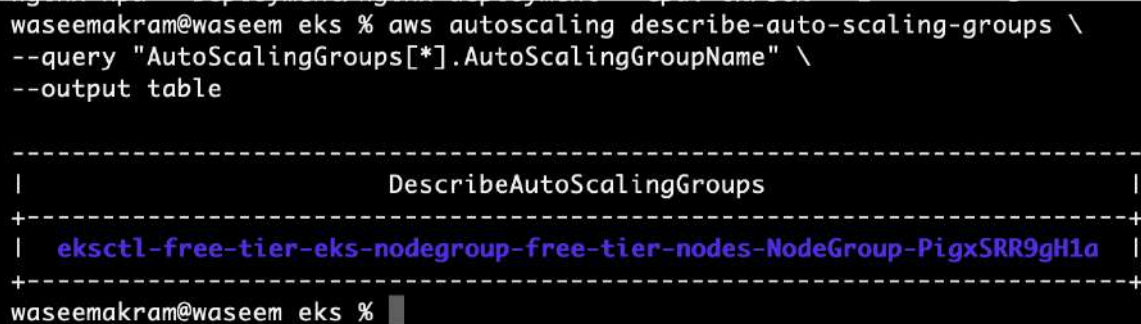
```
waseemakram@waseem eks % kubectl get hpa
```

TASK 4 – Setup Cluster Autoscaler

Step 1 – Get Auto Scaling Group Name

```
aws autoscaling describe-auto-scaling-groups \
--query "AutoScalingGroups[*].AutoScalingGroupName" \
--output table
```

Copy the ASG name.



```
waseemakram@waseem eks % aws autoscaling describe-auto-scaling-groups \
--query "AutoScalingGroups[*].AutoScalingGroupName" \
--output table
```

DescribeAutoScalingGroups
eksctl-free-tier-eks-nodegroup-free-tier-nodes-NodeGroup-PigxSRR9gH1a

```
waseemakram@waseem eks %
```

Step 2 – Add Required Tags to ASG

Replace:

- `<ASG_NAME>` → your ASG name
- `<cluster-name>` → free-tier-eks (your cluster name)

```
aws autoscaling create-or-update-tags --tags \
ResourceId=<ASG_NAME>,ResourceType=auto-scaling-
group,Key=k8s.io/cluster-autoscaler/
enabled,Value=true,PropagateAtLaunch=true \
ResourceId=<ASG_NAME>,ResourceType=auto-scaling-
group,Key=k8s.io/cluster-autoscaler/free-tier-
eks,Value=owned,PropagateAtLaunch=true
```

Replaced :

```
aws autoscaling create-or-update-tags --tags \
ResourceId=eksctl-free-tier-eks-nodegroup-free-tier-nodes-
NodeGroup-PigxSRR9gH1a,ResourceType=auto-scaling-
group,Key=k8s.io/cluster-autoscaler/
enabled,Value=true,PropagateAtLaunch=true \
ResourceId=eksctl-free-tier-eks-nodegroup-free-tier-nodes-
NodeGroup-PigxSRR9gH1a,ResourceType=auto-scaling-
group,Key=k8s.io/cluster-autoscaler/free-tier-
eks,Value=owned,PropagateAtLaunch=true
```

```
+-----+
waseemakram@waseem eks % aws autoscaling create-or-update-tags --tags \
ResourceId=eksctl-free-tier-eks-nodegroup-free-tier-nodes-NodeGroup-PigxSRR9gH1a,ResourceType=auto-scaling-group,Key=k8s.io/cluster-autoscaler/enabled,Value=true,PropagateAtLaun
ResourceId=eksctl-free-tier-eks-nodegroup-free-tier-nodes-NodeGroup-PigxSRR9gH1a,ResourceType=auto-scaling-group,Key=k8s.io/cluster-autoscaler/free-tier-eks,Value=owned,Propagat
true
waseemakram@waseem eks %
```

STEP 3 – Create IAM Policy

Go to:

AWS Console → IAM → Policies → Create Policy
Select JSON and paste this:

Step 1
● **Specify permissions**
○ Step 2
○ Review and create

Specify permissions [Info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "autoscaling:DescribeAutoScalingGroups",  
8         "autoscaling:DescribeAutoScalingInstances",  
9         "autoscaling:DescribeLaunchConfigurations",  
10        "autoscaling:DescribeScalingActivities",  
11        "autoscaling:SetDesiredCapacity",  
12        "autoscaling:TerminateInstanceInAutoScalingGroup",  
13        "ec2:DescribeLaunchTemplateVersions"  
14      ],  
15      "Resource": "*"   
16    }  
17  ]  
18 }  
19
```

Step 1
● Specify permissions
○ Step 2
● **Review and create** [Info](#)

Review and create [Info](#)

Review the permissions, specify details, and tags.

Policy details

Policy name
Enter a meaningful name to identify this policy.

Maximum 128 characters. Use alphanumeric and '+', '@', '_' characters.

Description - optional
Add a short explanation for this policy.

Maximum 1,000 characters. Use alphanumeric and '+', '@', '_' characters.

Permissions defined in this policy [Info](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), use the IAM console.

Allow (2 of 463 services)

Service	Access level	Resource	Request condition
EC2	Limited: List	All resources	None
EC2 Auto Scaling	Limited: List, Write	All resources	None

Add tags - optional [Info](#)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "autoscaling:DescribeAutoScalingGroups",
        "autoscaling:DescribeAutoScalingInstances",
        "autoscaling:DescribeLaunchConfigurations",
        "autoscaling:DescribeScalingActivities",
        "autoscaling:SetDesiredCapacity",
        "autoscaling:TerminateInstanceInAutoScalingGroup",
        "ec2:DescribeLaunchTemplateVersions"
      ],
      "Resource": "*"
    }
  ]
}
```

Create it.

STEP 4 – Create IAM Service Account

Replace YOUR_ACCOUNT_ID properly.

```
eksctl create iamserviceaccount \
--cluster free-tier-eks \
--namespace kube-system \
--name cluster-autoscaler \
--attach-policy-arn arn:aws:iam::<YOUR_ACCOUNT_ID>:policy/
AmazonEKSClusterAutoscalerPolicy \
--approve
```

Replaced :

```
eksctl create iamserviceaccount \  
--cluster free-tier-eks \  
--namespace kube-system \  
--name cluster-autoscaler \  
--attach-policy-arn arn:aws:iam::207662791773:policy/  
AmazonEKSClusterAutoscalerPolicy \  
--approve
```

```
waseemakram@waseem eks % eksctl utils associate-iam-oidc-provider \  
--region us-east-2 \  
--cluster free-tier-eks \  
--approve  
  
2026-02-28 16:31:25 [i] will create IAM Open ID Connect provider for cluster "free-tier-eks" in "us-east-2"  
2026-02-28 16:31:27 [✓] created IAM Open ID Connect provider for cluster "free-tier-eks" in "us-east-2"  
waseemakram@waseem eks % eksctl create iamserviceaccount \  
--cluster free-tier-eks \  
--namespace kube-system \  
--name cluster-autoscaler \  
--attach-policy-arn arn:aws:iam::207662791773:policy/AmazonEKSClusterAutoscalerPolicy \  
--approve  
  
2026-02-28 16:31:43 [i] 1 iamserviceaccount (kube-system/cluster-autoscaler) was included (based on the include/exclude rules)  
2026-02-28 16:31:43 [!] serviceaccounts that exist in Kubernetes will be excluded, use --override-existing-serviceaccounts to override  
2026-02-28 16:31:43 [i] 1 task: {  
  2 sequential sub-tasks: {  
    create IAM role for serviceaccount "kube-system/cluster-autoscaler",  
    create serviceaccount "kube-system/cluster-autoscaler",  
  } }  
2026-02-28 16:31:43 [i] building iamserviceaccount stack "eksctl-free-tier-eks-addon-iamserviceaccount-kube-system-cluster-autoscaler"  
2026-02-28 16:31:43 [i] deploying stack "eksctl-free-tier-eks-addon-iamserviceaccount-kube-system-cluster-autoscaler"  
2026-02-28 16:31:44 [i] waiting for CloudFormation stack "eksctl-free-tier-eks-addon-iamserviceaccount-kube-system-cluster-autoscaler"  
2026-02-28 16:32:15 [i] waiting for CloudFormation stack "eksctl-free-tier-eks-addon-iamserviceaccount-kube-system-cluster-autoscaler"  
2026-02-28 16:32:16 [i] created serviceaccount "kube-system/cluster-autoscaler"  
waseemakram@waseem eks %
```

STEP 5 – Install Cluster Autoscaler

Run this:

```
kubectl apply -f https://raw.githubusercontent.com/  
kubernetes/autoscaler/cluster-autoscaler-1.29.0/cluster-  
autoscaler/cloudprovider/aws/examples/cluster-autoscaler-  
autodiscover.yaml
```

You should see:

```
deployment.apps/cluster-autoscaler created
```

```
waseemakram@waseem eks % kubectl apply -f https://raw.githubusercontent.com/kubernetes/autoscaler/cluster-autoscaler-1.29.0/cluster-autodiscover.yaml

Warning: resource serviceaccounts/cluster-autoscaler is missing the kubectl.kubernetes.io/last-applied-configuration annotation which will be used on resources created declaratively by either kubectl create --save-config or kubectl apply. The missing annotation will be added to serviceaccount/cluster-autoscaler configured
clusterrole.rbac.authorization.k8s.io/cluster-autoscaler created
role.rbac.authorization.k8s.io/cluster-autoscaler created
clusterrolebinding.rbac.authorization.k8s.io/cluster-autoscaler created
rolebinding.rbac.authorization.k8s.io/cluster-autoscaler created
deployment.apps/cluster-autoscaler created
waseemakram@waseem eks %
```

STEP 6 – Edit Deployment

Now edit it:

```
kubectl -n kube-system edit deployment.apps/cluster-autoscaler
```

Inside the file find:

```
--cluster-name=
```

OR something like:

```
--cluster-name=<YOUR CLUSTER NAME>
```

Change it to:

```
--cluster-name=free-tier-eks
```

```
app: cluster-autoscaler
spec:
  containers:
  - command:
    - ./cluster-autoscaler
    - --v=4
    - --stderrthreshold=info
    - --cloud-provider=aws
    - --skip-nodes-with-local-storage=false
    - --expander=least-waste
    - --node-group-auto-discovery=asg:tag=k8s.io/cluster-autoscaler/enabled,k8s.io/cluster-autoscaler/free-tier-eks
    image: registry.k8s.io/autoscaling/cluster-autoscaler:v1.26.2
    imagePullPolicy: Always
  name: cluster-autoscaler
  resources:
    limits:
      cpu: 100m
      memory: 600Mi
    requests:
```

After saving, run:

```
kubectl get pods -n kube-system
```

Check if:

```
cluster-autoscaler-xxxxx    Running
```

```
waseemakram@waseem eks % kubectl -n kube-system edit deployment.apps/cluster-autoscaler
deployment.apps/cluster-autoscaler edited
waseemakram@waseem eks % kubectl get pods -n kube-system

NAME                                READY   STATUS    RESTARTS   AGE
aws-node-j7rb2                      2/2     Running   0           129m
aws-node-mmbsv                      2/2     Running   0           129m
cluster-autoscaler-6dfd6d6d85-5stvr 1/1     Running   0           10s
coredns-6b94694fcb-blkbh           1/1     Running   0           134m
coredns-6b94694fcb-f9pvr           1/1     Running   0           134m
kube-proxy-hgzdq                   1/1     Running   0           129m
kube-proxy-mjbvm                   1/1     Running   0           129m
metrics-server-6899488d4b-kw7pd     1/1     Running   0           129m
metrics-server-6899488d4b-qtt5b     1/1     Running   0           129m
waseemakram@waseem eks %
```

Then Check Logs (Very Important)

```
kubectl -n kube-system logs deployment/cluster-autoscaler
```

You should see something like:

```
Auto-discovered ASG: eksctl-free-tier-eks-nodegroup...
```

```
waseemakram@waseem eks % kubectl -n kube-system logs deployment/cluster-autoscaler
I0228 11:12:54.313425    1 flags.go:57] FLAG: --add-dir-header="false"
I0228 11:12:54.313499    1 flags.go:57] FLAG: --address=":8085"
I0228 11:12:54.313508    1 flags.go:57] FLAG: --alsologtostderr="false"
I0228 11:12:54.313512    1 flags.go:57] FLAG: --aws-use-static-instance-list="false"
I0228 11:12:54.313517    1 flags.go:57] FLAG: --balance-similar-node-groups="false"
I0228 11:12:54.313521    1 flags.go:57] FLAG: --balancing-ignore-label="[]"
I0228 11:12:54.313525    1 flags.go:57] FLAG: --balancing-label="[]"
I0228 11:12:54.313528    1 flags.go:57] FLAG: --cloud-config=""
I0228 11:12:54.313532    1 flags.go:57] FLAG: --cloud-provider="aws"
I0228 11:12:54.313535    1 flags.go:57] FLAG: --cloud-provider-gce-l7lb-src-cidrs="130.211.0.0/22,35.191.0.0/16"
I0228 11:12:54.313541    1 flags.go:57] FLAG: --cloud-provider-gce-lb-src-cidrs="130.211.0.0/22,209.85.152.0/22,209.85.204.0/22,35.191.0.0/22"
I0228 11:12:54.313547    1 flags.go:57] FLAG: --cluster-name=""
I0228 11:12:54.313557    1 flags.go:57] FLAG: --clusterapi-cloud-config-authoritative="false"
I0228 11:12:54.313562    1 flags.go:57] FLAG: --cordons-node-before-terminating="false"
I0228 11:12:54.313622    1 flags.go:57] FLAG: --cores-total="0:320000"
I0228 11:12:54.313704    1 flags.go:57] FLAG: --daemonset-eviction-for-empty-nodes="false"
I0228 11:12:54.313763    1 flags.go:57] FLAG: --daemonset-eviction-for-occupied-nodes="true"
I0228 11:12:54.313773    1 flags.go:57] FLAG: --debugging-snapshot-enabled="false"
I0228 11:12:54.313778    1 flags.go:57] FLAG: --emit-per-nodegroup-metrics="false"
I0228 11:12:54.313782    1 flags.go:57] FLAG: --enforce-node-group-min-size="false"
I0228 11:12:54.313786    1 flags.go:57] FLAG: --estimator="binpacking"
I0228 11:12:54.313835    1 flags.go:57] FLAG: --expander="least-waste"
```

FINAL TEST – Force New EC2 Node Creation

Right now you have 2 nodes.

We will create heavy pods so cluster has no space →

Pods go **Pending** →

Cluster Autoscaler creates new EC2 node automatically.

STEP 1 – Create Heavy Deployment

Create file:

```
vi test-deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-deployment
spec:
  replicas: 5
  selector:
    matchLabels:
      app: test
  template:
    metadata:
      labels:
        app: test
    spec:
      containers:
      - name: nginx
        image: nginx
        resources:
          requests:
            cpu: "700m"
            memory: "700Mi"
```

STEP 2 – Apply It

```
kubectl apply -f test-deployment.yaml
```

STEP 3 – Watch Pods

```
kubectl get pods -w
```

You should see:

Some pods → **Pending**

That means cluster has no capacity.

```
waseemakram@waseem eks % vi test-deployment.yaml
waseemakram@waseem eks % kubectl apply -f test-deployment.yaml
deployment.apps/test-deployment created
waseemakram@waseem eks % kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-66f5cbd4b-lrwtx	1/1	Running	0	49m
test-deployment-68c9569ff6-22bp4	0/1	Pending	0	2s
test-deployment-68c9569ff6-4wdtb	0/1	Pending	0	2s
test-deployment-68c9569ff6-6trc5	1/1	Running	0	3s
test-deployment-68c9569ff6-vkltg	0/1	Pending	0	3s
test-deployment-68c9569ff6-vrbpk	0/1	Pending	0	3s

STEP 4 – Watch Nodes

Open new terminal and run:

```
kubectl get nodes -w
```

```
waseemakram@waseem eks % kubectl get nodes -w
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-27-228.us-east-2.compute.internal	Ready	<none>	135m	v1.31.14-eks-efcacff
ip-192-168-32-33.us-east-2.compute.internal	Ready	<none>	135m	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	2s	v1.31.14-eks-efcacff

After 1–2 minutes...

You should see:

—-> A NEW node appears

That means:

- * Cluster Autoscaler created new EC2 instance
- * Infrastructure scaled automatically
- * Task 4 completed successfully

```
waseemakram@waseem eks % kubectl get nodes -w
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-27-228.us-east-2.compute.internal	Ready	<none>	135m	v1.31.14-eks-efcacff
ip-192-168-32-33.us-east-2.compute.internal	Ready	<none>	135m	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	0s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	2s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	NotReady	<none>	9s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	Ready	<none>	11s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	Ready	<none>	11s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	Ready	<none>	12s	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	Ready	<none>	30s	v1.31.14-eks-efcacff

```
waseemakram@waseem eks % kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-27-228.us-east-2.compute.internal	Ready	<none>	136m	v1.31.14-eks-efcacff
ip-192-168-32-33.us-east-2.compute.internal	Ready	<none>	136m	v1.31.14-eks-efcacff
ip-192-168-67-29.us-east-2.compute.internal	Ready	<none>	94s	v1.31.14-eks-efcacff

```
waseemakram@waseem eks %
```

Instances (3) Info

Find Instance by attribute or tag (case-sensitive) All states

running Clear filters

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv
☐	free-tier-eks-f...	i-0352f9d56b498e307	Running	t3.small	3/3 checks passed	View alarms +	us-east-2c	ec2-18-118-134.us-...	18.118.13
☐	free-tier-eks-f...	i-0050d605e1b9b8aa2	Running	t3.small	3/3 checks passed	View alarms +	us-east-2b	ec2-3-15-200-111.us-...	3.15.200.1
☐	free-tier-eks-f...	i-0983ccebdc001096d	Running	t3.small	3/3 checks passed	View alarms +	us-east-2a	ec2-3-135-200-203.us-...	3.135.200

What Is Happening Internally?

1. Pods request more CPU
2. No space in current nodes
3. Pods go Pending
4. Autoscaler detects unschedulable pods
5. It increases ASG desired capacity
6. AWS launches new EC2
7. Pods become Running

TASK 5 – Setup Job and CronJob

PART 1 – Create a Job (Runs Once)

◆ Step 1 – Create Job YAML

```
vi job.yaml
```

```
apiVersion: batch/v1
kind: Job
metadata:
  name: simple-job
spec:
  backoffLimit: 2
  template:
    spec:
      containers:
      - name: busybox
        image: busybox
        command: ["echo", "Hello from Kubernetes Job"]
        restartPolicy: Never
```

Step 2 – Apply Job

```
kubectl apply -f job.yaml
```

Step 3 – Verify

```
kubectl get jobs
kubectl get pods
You should see:
```

- Job → Completed
- Pod → Completed

```
waseemakram@waseem eks % kubectl apply -f job.yaml

job.batch/simple-job created
waseemakram@waseem eks % kubectl get jobs
NAME          STATUS    COMPLETIONS  DURATION  AGE
simple-job     Complete  1/1          5s        7s
waseemakram@waseem eks % kubectl get pods
NAME                                READY  STATUS    RESTARTS  AGE
nginx-deployment-66f5cbd4b-lrwtx    1/1    Running   0         58m
simple-job-pszkc                      0/1    Completed 0         14s
waseemakram@waseem eks %
```

Step 4 – Check Output

```
kubectl logs <pod-name>
```

```
simple-job-pszkc                      0/1    Completed 0         14s
waseemakram@waseem eks % kubectl logs simple-job-pszkc
Hello from Kubernetes Job
waseemakram@waseem eks %
```

PART 2 – Create a CronJob (Runs Automatically)

CronJob runs repeatedly based on schedule.

◆ Step 1 – Create CronJob YAML

```
vi cronjob.yaml
```

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: simple-cronjob
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: busybox
              image: busybox
              command: ["echo", "Hello from CronJob"]
          restartPolicy: Never
```

Step 2 – Apply CronJob

```
kubectl apply -f cronjob.yaml
```

Step 3 – Watch It

```
kubectl get cronjobs
kubectl get jobs
kubectl get pods
Every 1 minute:
```

- New Job created
- New Pod runs
- Status → Completed

```
waseemakram@waseem eks % kubectl apply -f cronjob.yaml
cronjob.batch/simple-cronjob created
waseemakram@waseem eks % kubectl get cronjobs
NAME                SCHEDULE    TIMEZONE    SUSPEND    ACTIVE    LAST SCHEDULE    AGE
simple-cronjob      */1 * * * *    <none>      False      0         <none>          5s
waseemakram@waseem eks % kubectl get jobs
NAME                STATUS      COMPLETIONS    DURATION    AGE
simple-cronjob-29537970    Running    0/1            2s          2s
simple-job             Complete    1/1            5s          3m20s
waseemakram@waseem eks % kubectl get pods
NAME                READY    STATUS      RESTARTS    AGE
nginx-deployment-66f5cbd4b-lrwtx    1/1      Running     0            61m
simple-cronjob-29537970-plclw         0/1      Completed   0            7s
simple-job-pszkc                     0/1      Completed   0            3m25s
waseemakram@waseem eks %
```

Difference (Important for Interview)

```
simple-job             Complete    1/1            5s          3m20s
waseemakram@waseem eks % kubectl get pods
NAME                READY    STATUS      RESTARTS    AGE
nginx-deployment-66f5cbd4b-lrwtx    1/1      Running     0            61m
simple-cronjob-29537970-plclw         0/1      Completed   0            7s
simple-job-pszkc                     0/1      Completed   0            3m25s
waseemakram@waseem eks % kubectl get pods
NAME                READY    STATUS      RESTARTS    AGE
nginx-deployment-66f5cbd4b-lrwtx    1/1      Running     0            63m
simple-cronjob-29537970-plclw         0/1      Completed   0            101s
simple-cronjob-29537971-8stkq         0/1      Completed   0            41s
simple-job-pszkc                     0/1      Completed   0            4m59s
waseemakram@waseem eks %
```

Job → Runs once

CronJob → Runs on schedule

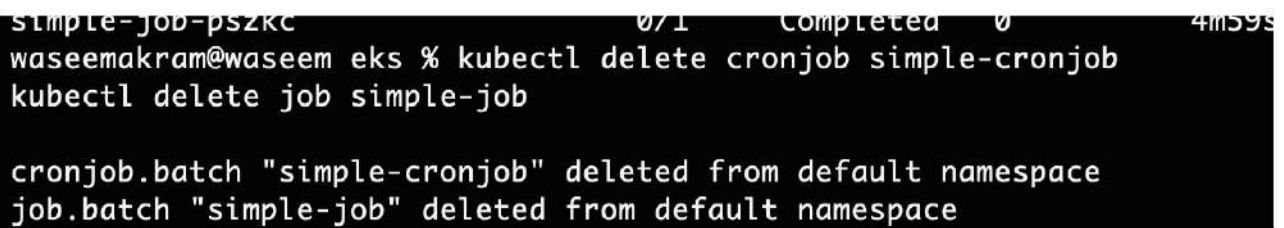
Task 5 Status

- ✓ Job Created
- ✓ CronJob Created
- ✓ Verified
- ✓ Logs Checked

Now Clean It (Optional but Recommended)

If you don't want repeated CronJobs:

```
kubectl delete cronjob simple-cronjob  
kubectl delete job simple-job
```

A terminal window with a black background and white text. The first line shows a pod status: 'simple-job-pszkc 0/1 Completed 0 4m59s'. The second line shows the user 'waseemakram@waseem' in the 'eks' namespace running 'kubectl delete cronjob simple-cronjob'. The third line shows the command 'kubectl delete job simple-job'. The fourth line shows the output 'cronjob.batch "simple-cronjob" deleted from default namespace'. The fifth line shows the output 'job.batch "simple-job" deleted from default namespace'.

```
simple-job-pszkc 0/1 Completed 0 4m59s  
waseemakram@waseem eks % kubectl delete cronjob simple-cronjob  
kubectl delete job simple-job  
  
cronjob.batch "simple-cronjob" deleted from default namespace  
job.batch "simple-job" deleted from default namespace
```

Task 6 –Create secret and inject inside pod

Secrets in Kubernetes.

This is very important for interviews.

We will do 3 things:

1. Create Secret
2. Use Secret inside Pod
3. Verify inside container

STEP 1 – Create a Secret

We will create a simple username/password secret.

```
kubectl create secret generic db-secret \
--from-literal=username=admin \
--from-literal=password=admin123
```

Verify Secret

```
kubectl get secrets
```

To see details:

```
kubectl describe secret db-secret
```

You will see:

- username
- password

```
waseemakram@waseem eks % kubectl create secret generic db-secret \
--from-literal=username=admin \
--from-literal=password=admin123

secret/db-secret created
waseemakram@waseem eks % kubectl get secrets

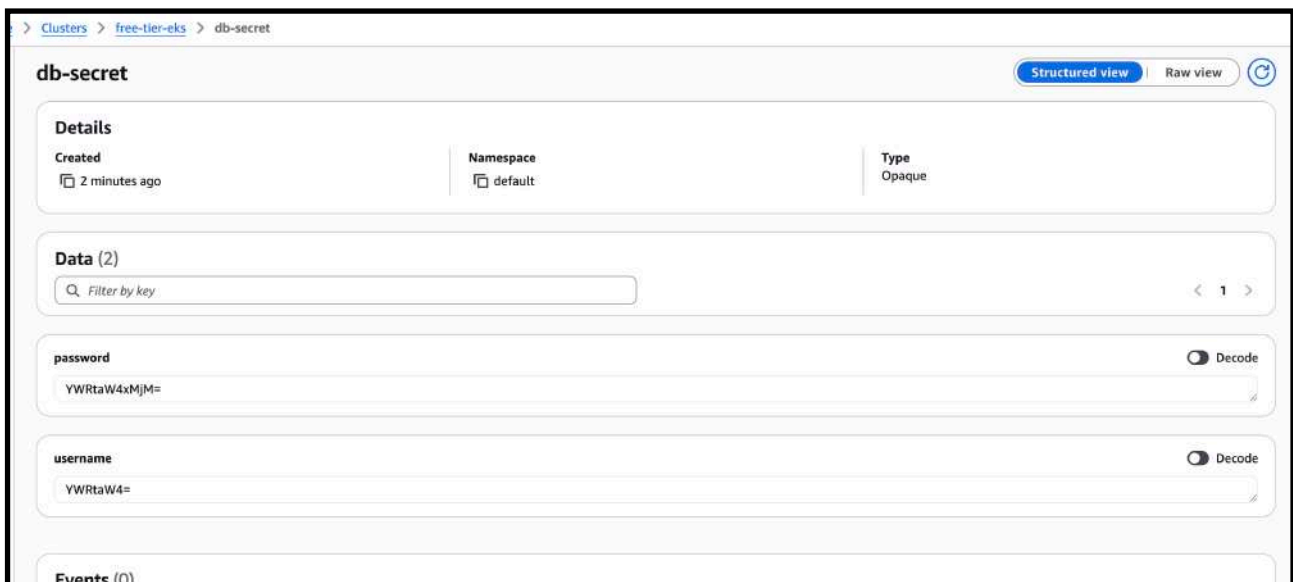
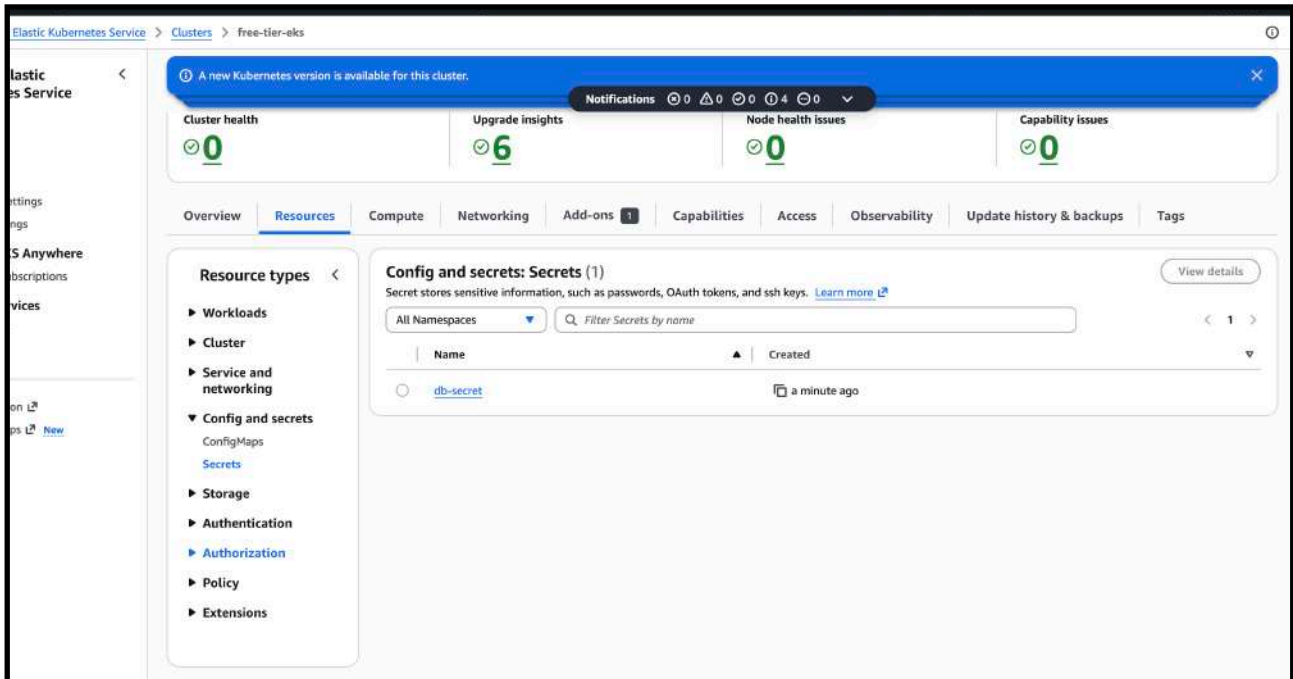
NAME          TYPE      DATA   AGE
db-secret     Opaque    2        7s
waseemakram@waseem eks % kubectl describe secret db-secret

Name:          db-secret
Namespace:     default
Labels:        <none>
Annotations:   <none>

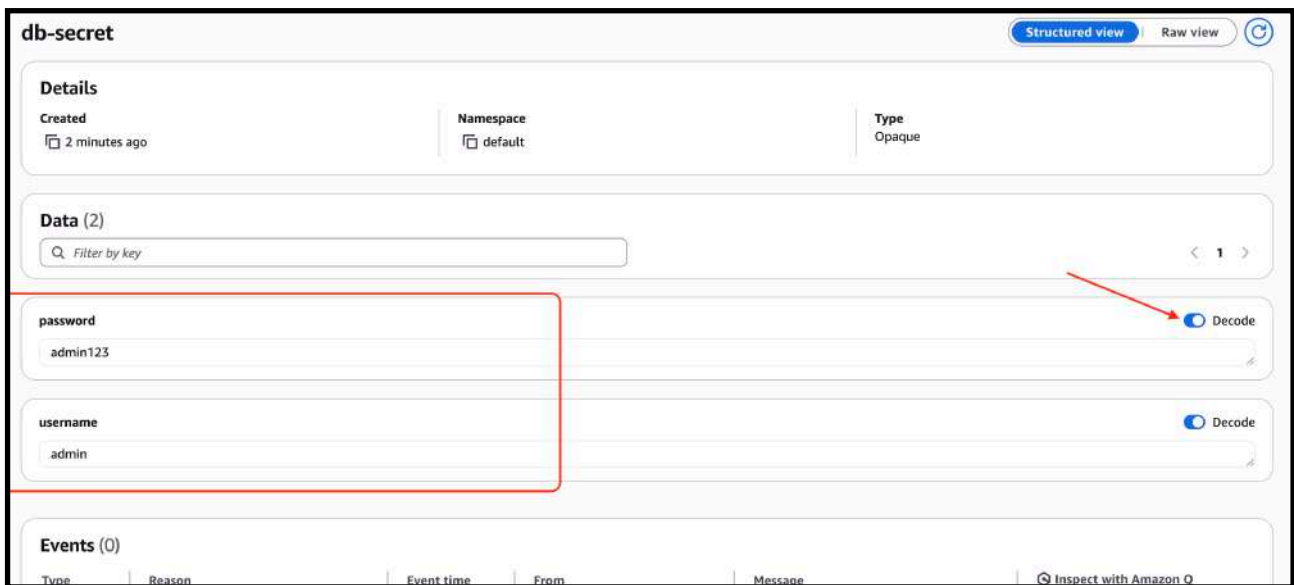
Type: Opaque

Data
====
password: 8 bytes
username: 5 bytes
waseemakram@waseem eks %
```

—> now check in cluster



—> we are able to see details by decord



STEP 2 – Create Pod Using Secret

Now create pod YAML.

```
vi secret-pod.yaml
```

```
waseemakram@waseem eks % vi secret-pod.yaml
waseemakram@waseem eks % cat secret-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-pod
spec:
  containers:
  - name: nginx
    image: nginx
    env:
    - name: DB_USERNAME
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: username
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: password
waseemakram@waseem eks %
```

STEP 3 – Apply Pod

```
kubectl apply -f secret-pod.yaml
```

Check:

```
kubectl get pods
```

```
waseemakram@waseem eks % kubectl apply -f secret-pod.yaml
pod/secret-pod created
waseemakram@waseem eks % kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-66f5cbd4b-lrwtx	1/1	Running	0	76m
secret-pod	1/1	Running	0	5s

```
waseemakram@waseem eks %
```

STEP 4 – Verify Secret Inside Pod

Enter container:

```
kubectl exec -it secret-pod -- /bin/bash
```

Inside container:

```
echo $DB_USERNAME
echo $DB_PASSWORD
```

```
secret-pod 1/1 Running 0s
waseemakram@waseem eks % kubectl exec -it secret-pod -- /bin/bash

root@secret-pod:/# echo $DB_USERNAME
echo $DB_PASSWORD
admin
admin123
root@secret-pod:/#
```

You have successfully:

- * Created Secret
- * Injected into Pod
- * Accessed inside container
- * Verified username & password

Task 7 – Check different add-ons available on EKS.

What are EKS Add-ons?

Add-ons are components like:

- VPC CNI
- CoreDNS
- kube-proxy
- EBS CSI driver
- etc.

They extend cluster functionality.

STEP 1 – List Available Add-ons (From AWS)

Run:

```
aws eks describe-addon-versions --region us-east-2 --output table
```

This shows all supported add-ons for your region.

```
waseemakram@waseem eks % aws eks describe-addon-versions --region us-east-2 --output table
```

DescribeAddonVersions				
addons				
addonName	defaultNamespace	owner	publisher	type
adot	opentelemetry-operator-system	aws	eks	observability
addonVersions				
addonVersion	requiresConfiguration	requiresIamPermissions		
v0.141.0-eksbuild.1	False	True		
architecture				
amd64				
arm64				
compatibilities				
clusterVersion	defaultVersion			
1.35	True			
platformVersions				
*				
compatibilities				
clusterVersion	defaultVersion			
1.34	True			
platformVersions				
*				

STEP 2 – Check Installed Add-ons in Your Cluster

```
aws eks list-addons --cluster-name free-tier-eks --region us-east-2
```

You will see something like:

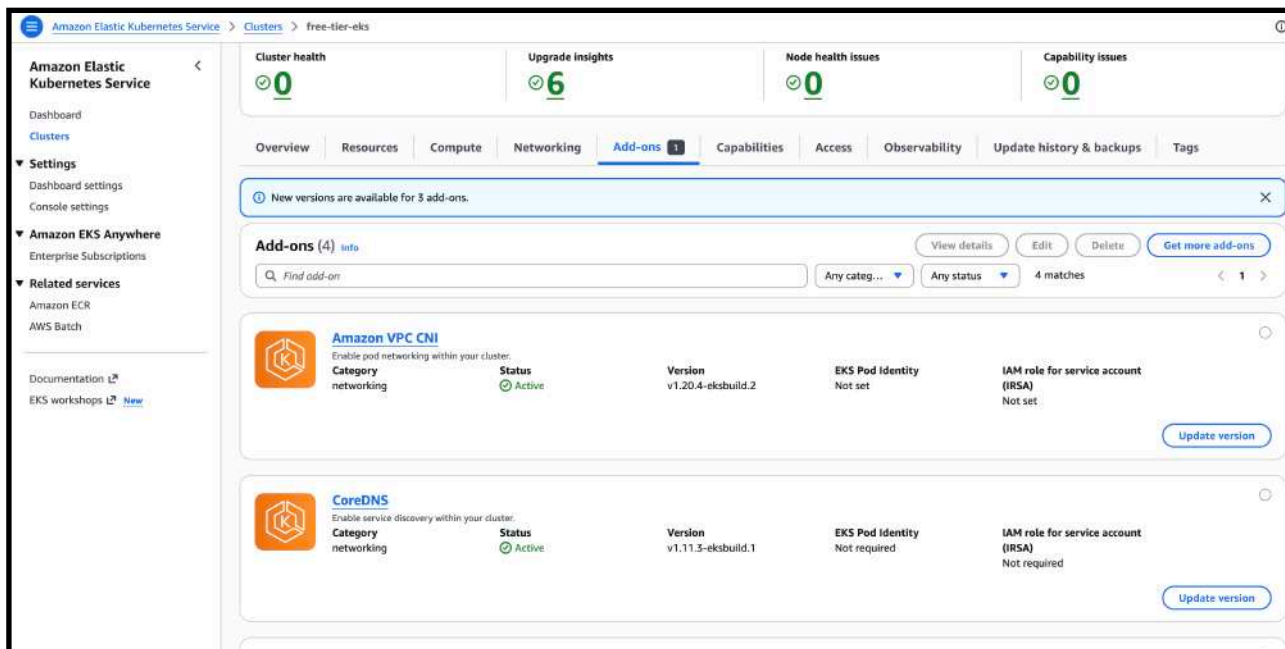
```
waseemakram@waseem eks % aws eks list-addons --cluster-name free-tier-eks --region us-east-2
{
  "addons": [
    "coredns",
    "kube-proxy",
    "metrics-server",
    "vpc-cni"
  ]
}
waseemakram@waseem eks %
```

```
waseemakram@waseem eks % kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
aws-node-j7rb2	2/2	Running	0	171m
aws-node-mmbsv	2/2	Running	0	171m
cluster-autoscaler-6dfd6d6d85-5stvr	1/1	Running	0	41m
coredns-6b94694fcb-blkbh	1/1	Running	0	176m
coredns-6b94694fcb-f9pvr	1/1	Running	0	176m
kube-proxy-hgzdq	1/1	Running	0	171m
kube-proxy-mjbvm	1/1	Running	0	171m
metrics-server-6899488d4b-kw7pd	1/1	Running	0	170m
metrics-server-6899488d4b-qtt5b	1/1	Running	0	170m

```
waseemakram@waseem eks %
```

--> go and check your cluster add ons in console



Task 8 – Upgrade EKS Cluster

IMPORTANT

EKS upgrade has 2 parts:

- 1 Control Plane Upgrade
- 2 Node Group Upgrade

Both must match Kubernetes version.

STEP 1 – Check Current Version

```
aws eks describe-cluster \
  --name free-tier-eks \
  --region us-east-2 \
  --query "cluster.version"
```

Also check nodes:

kubectl get nodes

```
waseemakram@waseem eks % aws eks describe-cluster \
--name free-tier-eks \
--region us-east-2 \
--query "cluster.version"

"1.31"
waseemakram@waseem eks % kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-27-228.us-east-2.compute.internal	Ready	<none>	173m	v1.31.14-eks-efcacff
ip-192-168-32-33.us-east-2.compute.internal	Ready	<none>	173m	v1.31.14-eks-efcacff

```
waseemakram@waseem eks %
```

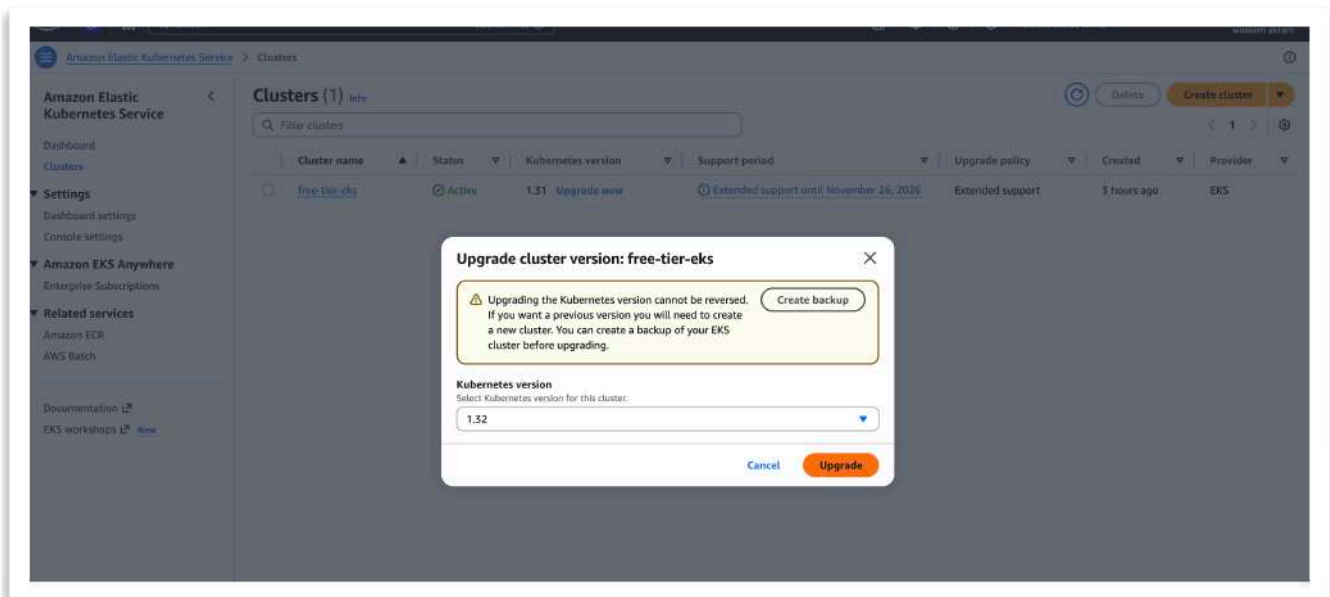
STEP 2 – Check Available Versions

aws eks describe-cluster-versions --region us-east-2

```
waseemakram@waseem eks % aws eks describe-cluster-versions --region us-east-2
{
  "clusterVersions": [
    {
      "clusterVersion": "1.35",
      "clusterType": "eks",
      "defaultPlatformVersion": "eks.5",
      "defaultVersion": true,
      "releaseDate": "2026-01-27T05:30:00+05:30",
      "endOfStandardSupportDate": "2027-03-27T05:30:00+05:30",
      "endOfExtendedSupportDate": "2028-03-27T05:30:00+05:30",
      "status": "STANDARD_SUPPORT",
      "versionStatus": "STANDARD_SUPPORT",
      "kubernetesPatchVersion": "1.35.0"
    },
    {
      "clusterVersion": "1.34",
      "clusterType": "eks",
      "defaultPlatformVersion": "eks.15",
      "defaultVersion": false,
      "releaseDate": "2025-10-02T05:30:00+05:30",
      "endOfStandardSupportDate": "2026-12-02T05:30:00+05:30",
      "endOfExtendedSupportDate": "2027-12-02T05:30:00+05:30",
      "status": "STANDARD_SUPPORT",
      "versionStatus": "STANDARD_SUPPORT",
      "kubernetesPatchVersion": "1.34.4"
    },
    {
      "clusterVersion": "1.33",
      "clusterType": "eks",
      "defaultPlatformVersion": "eks.29",
      "defaultVersion": false,
      "releaseDate": "2025-05-29T05:30:00+05:30",
      "endOfStandardSupportDate": "2026-07-29T05:30:00+05:30",
      "endOfExtendedSupportDate": "2027-07-29T05:30:00+05:30",
      "status": "STANDARD_SUPPORT",
      "versionStatus": "STANDARD_SUPPORT",
      "kubernetesPatchVersion": "1.33.8"
    },
    {
      "clusterVersion": "1.32",
      "clusterType": "eks",
      "defaultPlatformVersion": "eks.36",
      "defaultVersion": false,
      "releaseDate": "2025-01-23T05:30:00+05:30",
      "endOfStandardSupportDate": "2026-03-23T05:30:00+05:30",
      "endOfExtendedSupportDate": "2027-03-23T05:30:00+05:30",
      "status": "STANDARD_SUPPORT",
      "versionStatus": "STANDARD_SUPPORT",
      "kubernetesPatchVersion": "1.32.12"
    },
    {
      "clusterVersion": "1.31",

```

We can update from console also



We going to update from terminal

STEP 1 — Upgrade Control Plane

Run this:

```
aws eks update-cluster-version \  
--name free-tier-eks \  
--kubernetes-version 1.32 \  
--region us-east-2
```

```
waseemakram@waseem eks % aws eks update-cluster-version \  
--name free-tier-eks \  
--kubernetes-version 1.32 \  
--region us-east-2  
  
{  
  "update": {  
    "id": "95183ca3-f74a-312e-bfb1-7a197e46cc14",  
    "status": "InProgress",  
    "type": "VersionUpdate",  
    "params": [  
      {  
        "type": "Version",  
        "value": "1.32"  
      },  
      {  
        "type": "PlatformVersion",  
        "value": "eks.36"  
      }  
    ],  
    "createdAt": "2026-02-28T17:37:17.934000+05:30",  
    "errors": []  
  }  
}
```

Copy the "id" value.

STEP 2 — Monitor Upgrade

Run this to track progress:

```
aws eks describe-update \  
--name free-tier-eks \  
--region us-east-2 \  
--update-id 95183ca3-f74a-312e-bfb1-7a197e46cc14 \  
--query "update.status" \  
Keep running every 30–40 seconds.
```

```
waseemakram@waseem eks % aws eks describe-update \  
--name free-tier-eks \  
--region us-east-2 \  
--update-id 95183ca3-f74a-312e-bfb1-7a197e46cc14 \  
--query "update.status" \  
  
"InProgress"   
waseemakram@waseem eks % aws eks describe-update \  
--name free-tier-eks \  
--region us-east-2 \  
--update-id 95183ca3-f74a-312e-bfb1-7a197e46cc14 \  
--query "update.status"   
  
"InProgress"   
waseemakram@waseem eks % aws eks describe-update \  
--name free-tier-eks \  
--region us-east-2 \  
--update-id 95183ca3-f74a-312e-bfb1-7a197e46cc14 \  
--query "update.status"   
  
"InProgress"   
waseemakram@waseem eks % aws eks describe-cluster \  
--name free-tier-eks \  
--region us-east-2 \  
--query "cluster.status"   
  
"UPDATING"   
waseemakram@waseem eks % █
```

Everything is correct.

Right now:

- `update.status` → **InProgress**
- `cluster.status` → **UPDATING**

That means control plane is upgrading properly.

Now Just Wait

This usually takes **5–10 minutes**.

Keep checking with:

```
aws eks describe-cluster \
--name free-tier-eks \
--region us-east-2 \
--query "cluster.status"
```

When it shows:

"ACTIVE"

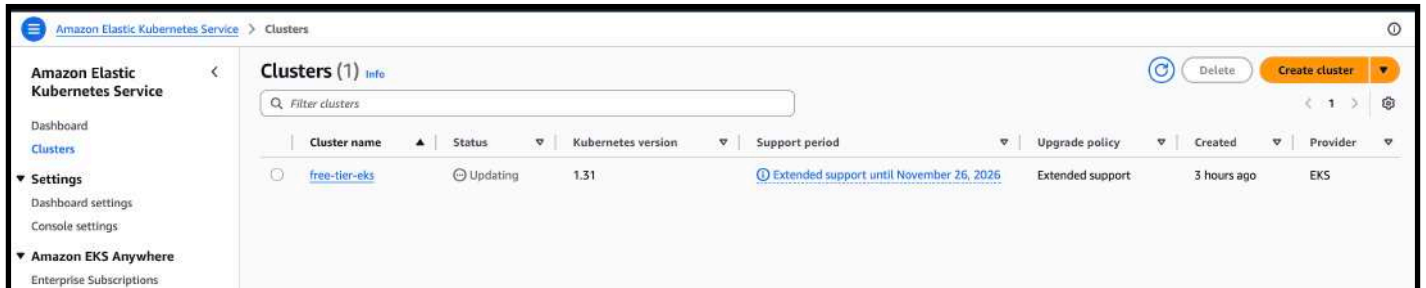
Then check:

```
aws eks describe-update \
--name free-tier-eks \
--region us-east-2 \
--update-id 95183ca3-f74a-312e-bfb1-7a197e46cc14 \
--query "update.status"
```

It should show:

"Successful"

—> check cluster status in console is updating



—> it will take 5-10 min



—> now we can see active and cluster is to 1.32 version

—-> check in terminal

```
waseemakram@waseem eks % aws eks describe-cluster \
--name free-tier-eks \
--region us-east-2 \
--query "cluster.status"

"ACTIVE"
waseemakram@waseem eks % aws eks describe-update \
--name free-tier-eks \
--region us-east-2 \
--update-id 95183ca3-f74a-312e-bfb1-7a197e46cc14 \
--query "update.status"

"Successful"
waseemakram@waseem eks %
```

NEXT STEP — Upgrade Node

```
waseemakram@waseem eks % kubectl get nodes -o wide
```

NAME	CONTAINER-RUNTIME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
ip-192-168-27-228.us-east-2.compute.internal	containerd://2.1.5	Ready	<none>	3h15m	v1.31.14-eks-efcacff	192.168.27.228	18.118.134.38	Amazon Linux 2023.10.20260216
ip-192-168-32-33.us-east-2.compute.internal	containerd://2.1.5	Ready	<none>	3h15m	v1.31.14-eks-efcacff	192.168.32.33	3.135.200.203	Amazon Linux 2023.10.20260216

```
waseemakram@waseem eks %
```

Upgrade EKS Worker Nodes from 1.31 to 1.32

Step 1: Verify Current Cluster Version

```
aws eks describe-cluster \
--name free-tier-eks \
--region us-east-2 \
--query "cluster.version"
Output:
```

"1.31"

Step 2: Upgrade Control Plane

```
aws eks update-cluster-version \  
--name free-tier-eks \  
--kubernetes-version 1.32 \  
--region us-east-2
```

Check status:

```
aws eks describe-cluster \  
--name free-tier-eks \  
--region us-east-2 \  
--query "cluster.status"
```

Wait until:

ACTIVE

Step 3: Verify Node Version Before Upgrade

```
kubectl get nodes
```

Output:

v1.31.x-eks-xxxxx

Step 4: Upgrade Nodegroup (Managed Nodegroup)

```
eksctl upgrade nodegroup \  
--cluster free-tier-eks \  
--name free-tier-nodes \  
--region us-east-2
```

This performs:

- Rolling update
- Launch new 1.32 nodes
- Drain old 1.31 nodes
- Terminate old nodes

Step 5: Verify Node Upgrade

```
kubectl get nodes
```

Output:

```
v1.32.x-eks-xxxxx
```

Final Result

- Control plane upgraded to 1.32
- Worker nodes upgraded to 1.32
- Cluster running successfully

```
waseemakram@waseem eks % kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-20-90.us-east-2.compute.internal	Ready	<none>	105s	v1.32.12-eks-efcacff
ip-192-168-69-15.us-east-2.compute.internal	Ready	<none>	108s	v1.32.12-eks-efcacff

```
waseemakram@waseem eks %
```

Always upgrade control plane first

Then upgrade nodegroups

Downgrade is not supported in EKS

Upgrade happens in rolling manner without downtime

